

# **QUANTUM-ENHANCED MULTI-MODAL VECTOR SEARCH: A HYBRID DATABASE WEB APP**



Elhan Mahmutović

Adna Kikanović

Adnan Mušanović

Mirza Abdulahović

INTERNATIONAL UNIVERSITY OF SARAJEVO

2026

# **QUANTUM-ENHANCED MULTI-MODAL VECTOR SEARCH: A HYBRID DATABASE WEB APP**

BY

Elhan Mahmutović – Software Engineering

Adna Kikanović – Software Engineering

Adnan Mušanović – Software Engineering

Mirza Abdulahović – Software Engineering

A graduation project submitted in partial fulfillment of the  
requirements for the degree of Bachelor of Science at the  
Faculty of Engineering and Natural Sciences  
International University of Sarajevo

June 2026

## APPROVAL PAGE

I certify that I have supervised and read this study and that in my opinion, it conforms to acceptable standards of scholarly presentation and is fully adequate, in scope and quality, as a graduation project for the degree of Bachelor of Science at the Faculty of Engineering and Natural Sciences.

.....  
Assist. Prof. Dr. Ali Abd Almisreb

*Mentor*

## DECLARATION

We hereby declare that all information in this document has been obtained and presented in accordance with academic rules and ethical conduct. We also declare that, as required by these rules and conduct, we have fully cited and referenced all material and results that are not original to this work.

Elhan Mahmutović

Signature .....

Adna Kikanović

Signature .....

Adnan Mušanović

Signature .....

Mirza Abdulahović

Signature .....

Date .....

INTERNATIONAL UNIVERSITY OF SARAJEVO

# DECLARATION OF COPYRIGHT AND AFFIRMATION OF FAIR USE OF UNPUBLISHED WORK

Copyright © 2026 by Elhan Mahmutović, Adna Kikanović, Adnan Mušanović, and Mirza Abdulahović. All rights reserved.

## Quantum-Enhanced Multi-Modal Vector Search: A Hybrid Database Web App

No part of this unpublished work may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording or otherwise without prior written permission of the copyright holder and IUS Library.

Affirmed by:

Elhan Mahmutović

Signature .....

Adna Kikanović

Signature .....

Adnan Mušanović

Signature .....

Mirza Abdulahović

Signature .....

Date .....

## **ACKNOWLEDGEMENTS**

We thank our supervisor, Assist. Prof. Dr. Ali Abd Almisreb, for his guidance, patience, and feedback throughout this project. We are grateful to the Faculty of Engineering and Natural Sciences at the International University of Sarajevo for the teaching and resources that made this work possible. Finally, we thank our families and friends for their encouragement and support.

## LIST OF ABBREVIATIONS

|         |                                           |
|---------|-------------------------------------------|
| AI      | Artificial Intelligence                   |
| API     | Application Programming Interface         |
| CLIP    | Contrastive Language – Image Pre-training |
| CPU     | Central Processing Unit                   |
| FAISS   | Facebook AI Similarity Search             |
| GPU     | Graphics Processing Unit                  |
| HNSW    | Hierarchical Navigable Small World        |
| IBM QPU | IBM Quantum Processing Unit               |
| MRR     | Mean Reciprocal Rank                      |
| NISQ    | Noisy Intermediate-Scale Quantum          |
| qRAM    | Quantum Random-Access Memory              |
| SDG     | Sustainable Development Goal              |
| SQL     | Structured Query Language                 |
| ViT     | Vision Transformer                        |

# TABLE OF CONTENTS

|                                                      |            |
|------------------------------------------------------|------------|
| <b>APPROVAL PAGE</b> . . . . .                       | <b>iii</b> |
| <b>DECLARATION</b> . . . . .                         | <b>iv</b>  |
| <b>DECLARATION OF COPYRIGHT</b> . . . . .            | <b>v</b>   |
| <b>ACKNOWLEDGEMENTS</b> . . . . .                    | <b>vi</b>  |
| <b>LIST OF ABBREVIATIONS</b> . . . . .               | <b>vii</b> |
| <b>ABSTRACT</b> . . . . .                            | <b>x</b>   |
| <b>1 INTRODUCTION</b> . . . . .                      | <b>1</b>   |
| 1.1 Background of Study . . . . .                    | 1          |
| 1.2 Problem Statement and Research Aims . . . . .    | 3          |
| 1.3 Research Questions and Objectives . . . . .      | 3          |
| <b>2 LITERATURE REVIEW</b> . . . . .                 | <b>5</b>   |
| 2.1 Classical Vector Search . . . . .                | 5          |
| 2.2 Cross-Modal Embeddings . . . . .                 | 5          |
| 2.3 Quantum Similarity Primitives . . . . .          | 6          |
| 2.4 Hybrid Classical–Quantum Architectures . . . . . | 6          |
| <b>3 RESEARCH METHODOLOGY</b> . . . . .              | <b>8</b>   |
| 3.1 System Overview . . . . .                        | 8          |
| 3.2 Methodology Diagram . . . . .                    | 8          |
| 3.3 Dataset and Ground Truth . . . . .               | 9          |
| 3.4 Embedding Pipeline . . . . .                     | 9          |
| 3.5 Search Engines . . . . .                         | 10         |
| 3.5.1 Classical Baselines . . . . .                  | 10         |
| 3.5.2 Quantum Engines . . . . .                      | 10         |
| 3.6 Evaluation Metrics . . . . .                     | 12         |
| 3.7 Experimental Configuration . . . . .             | 13         |



|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| Use of AI Tools . . . . .                                           | 13        |
| <b>4 RESULTS AND ANALYSIS . . . . .</b>                             | <b>17</b> |
| 4.1 Accuracy Across Engines . . . . .                               | 17        |
| 4.2 Quantum Resource Cost . . . . .                                 | 18        |
| 4.3 Wall-Clock Behaviour (Within-Engine Only) . . . . .             | 21        |
| 4.4 IBM Hardware Sanity Check . . . . .                             | 21        |
| 4.5 Discussion . . . . .                                            | 22        |
| 4.6 Limitations . . . . .                                           | 23        |
| <b>5 SUSTAINABILITY, INCLUSIVITY, AND SOCIETAL IMPACT . . . . .</b> | <b>24</b> |
| <b>6 CONCLUSION . . . . .</b>                                       | <b>25</b> |
| <b>REFERENCES . . . . .</b>                                         | <b>26</b> |
| <b>A APPENDICES . . . . .</b>                                       | <b>27</b> |
| Appendix A: Engine Configuration . . . . .                          | 27        |
| Appendix B: Database Schema . . . . .                               | 28        |
| Appendix C: Source Code Repository . . . . .                        | 28        |

# ABSTRACT

## QUANTUM-ENHANCED MULTI-MODAL VECTOR SEARCH: A HYBRID DATABASE WEB APP

Vector search finds stored items whose numerical representations, called embeddings, are closest to a query. It is used in systems such as text-to-image retrieval, where a caption is used to find matching images. Quantum algorithms such as Grover’s search and the swap test suggest possible speed-ups for this kind of similarity search, but their practical value depends on data loading, hardware noise, and comparison with strong classical indexes.

We built a benchmark framework in which seven engines answer the same queries over the same CLIP (Contrastive Language-Image Pre-training) embeddings. The engines include exact classical search, approximate graph search, quantum similarity and search variants, and a hybrid graph-search plus quantum-reranking method. We tested them on a fixed twenty-image dataset across embedding dimensions and measurement shot counts. Accuracy was measured with Mean Reciprocal Rank, while cost was measured with comparisons, oracle calls, qubits, and circuit depth. The quantum engines ran mainly on a noiseless Qiskit simulator; one small IBM hardware run was used only to check that the execution path works on a real quantum processor.

The measured results do not show a practical quantum advantage for this task. Accuracy mainly followed embedding dimension, not whether the engine was quantum. The swap test matched the classical ranking at dimension 256 when enough measurements were used, and Grover matched its expected oracle-call count at the tested size. However, no quantum search engine beat the classical graph-search baseline. Grover’s apparent speed-up counts only the search after the data is already available to the oracle; building that oracle or loading the data would add back the main retrieval cost. We conclude that cross-modal vector search is mainly a data-access problem, not a task where quantum computation is the bottleneck, under current methods and hardware.

**Keywords:** *quantum computing, vector search, similarity search, swap test, Grover’s algorithm, CLIP, benchmarking*

# LIST OF TABLES

|          |                                                        |    |
|----------|--------------------------------------------------------|----|
| Table 1: | Algorithm cost summary . . . . .                       | 2  |
| Table 2: | Benchmark configuration grid. . . . .                  | 13 |
| Table 3: | Mean Reciprocal Rank by engine and dimension . . . . . | 17 |
| Table 4: | Operations per query and complexity class . . . . .    | 19 |
| Table 5: | Quantum circuit width and depth . . . . .              | 20 |
| Table 6: | Mean query time by engine and dimension . . . . .      | 21 |

# LIST OF FIGURES

|           |                                         |    |
|-----------|-----------------------------------------|----|
| Figure 1: | End-to-end system pipeline . . . . .    | 14 |
| Figure 2: | Experimental methodology flow . . . . . | 15 |
| Figure 3: | Swap-test circuit . . . . .             | 16 |
| Figure 4: | Grover circuit, one iteration . . . . . | 16 |
| Figure 5: | Grover oracle-call scaling . . . . .    | 20 |

# 1. INTRODUCTION

Modern machine-learning systems represent text, images, and other media as high-dimensional vectors called embeddings. Items with similar meaning are placed near one another, so searching a collection becomes a nearest-vector problem. This is vector search, the operation behind image search, recommendation, and retrieval-augmented language models. When the query and stored items are different media, such as text-to-image search, the task is cross-modal retrieval.

Quantum computers process information with qubits, whose states can interfere with one another. This is sometimes described as trying many answers at once, but that description is too simple: a measurement still returns one outcome, and the algorithm must make wrong answers less likely and useful answers more likely. Classical computers can also test many candidates in parallel when enough processors are available. A useful quantum speed-up therefore depends on the whole workflow, including data loading and measurement, not only on the quantum circuit itself [1].

Two quantum building blocks look relevant to vector search. Grover’s algorithm finds a marked item in an unstructured collection using far fewer queries than a classical scan, and the swap test estimates how similar two quantum states are. This study asks, through measurement rather than assertion, whether those building blocks are of practical use for cross-modal vector search at the scale and on the hardware available today. It is an investigation of where quantum search does and does not fit, not an attempt to demonstrate a quantum advantage.

## 1.1 Background of Study

For cross-modal retrieval we use CLIP (Contrastive Language-Image Pre-training), a model that turns images and captions into vectors in the same space [2]. Once both media are represented there, a query is answered by ranking image vectors by closeness to the text vector. Exhaustive search compares the query with every stored vector and costs  $O(N)$  comparisons for  $N$  items. Facebook AI Similarity Search (FAISS) implements this exact scan efficiently [3], while approximate methods such as the Hierarchical Navigable Small World (HNSW) graph reach a near neighbour in roughly  $O(\log N)$  graph steps [4].

The quantum side of the comparison rests on two ideas. Grover’s algorithm searches an unstructured collection of  $N$  items for a marked one using only about  $\sqrt{N}$  queries to an oracle, a quadratic improvement over the linear classical scan [5]. In this context, an oracle is the part of the algorithm that can recognise whether a candidate is the desired answer. The swap test takes two quantum states and estimates the squared overlap between them from a single measured qubit [6]; because a normalised embedding can be encoded in the amplitudes of a quantum state, that overlap is a similarity score. Grover can therefore be read as a search method, and the swap test as a similarity comparison.

Table 1 summarises the main algorithms before the detailed methodology. The important point is that Grover’s  $O(\sqrt{N})$  search is better than a classical exact scan, but not automatically better than HNSW, which is already roughly logarithmic because it accepts an approximate nearest neighbour.

**Table 1:** High-level cost summary for the search methods used in the project.  $N$  is the number of stored vectors and  $M$  is the hybrid shortlist size. The quantum costs count the search or comparison step only; data loading is discussed below.

| Method             | Cost                       | Meaning in this project                                                             |
|--------------------|----------------------------|-------------------------------------------------------------------------------------|
| Brute-force cosine | $O(N)$                     | Exact baseline; reads every stored vector.                                          |
| FAISS flat         | $O(N)$                     | Same exact scan implemented by FAISS.                                               |
| HNSW               | $O(\log N)$ approx.        | Fast approximate graph index; can miss the exact nearest neighbour.                 |
| Swap test          | $O(N)$ circuits            | Quantum similarity estimate; a full ranking still compares against every candidate. |
| Grover             | $O(\sqrt{N})$ oracle calls | Faster than an exact scan on paper, but only after a useful oracle exists.          |
| Hybrid             | $O(\log N + M)$            | HNSW searches; the swap test reranks a shortlist of $M$ candidates.                 |

The difficulty is that these speed-ups assume the data is already available inside the quantum computer. For Grover to search real vectors, an oracle would have to recognise the closest item while the whole dataset is held in superposition, which requires a quantum random-access memory that does not yet exist at usable scale; loading the data one item at a time instead costs  $O(N)$  and removes the very advantage the algorithm promises [7]. This tension between an algorithm’s asymptotic promise and the cost of getting data into the machine is the central question the study examines.

## 1.2 Problem Statement and Research Aims

Classical vector search is mature and fast. Whether quantum methods add anything at today’s scale is less clear, especially because practical-advantage analyses warn that input and output costs can remove the benefit of algorithms with only quadratic speed-ups [1]. The gap this work addresses is empirical: there are few like-for-like comparisons that run classical and quantum similarity search on the same data while reporting both retrieval accuracy and resource cost. We built such a comparison. A common interface lets classical, quantum, and hybrid engines answer the same queries over the same embeddings, so their rankings are scored identically.

We frame the work as a measurement rather than a proof of advantage. Present-day devices are small and noisy, the regime usually called noisy intermediate-scale quantum (NISQ), so we run the quantum engines mostly on a classical simulator and validate the hardware path with a single small run on a real quantum processor. The goal is to determine, on a controlled cross-modal dataset, how the quantum and hybrid engines compare with strong classical baselines, and to state plainly the conditions under which each approach is preferable.

## 1.3 Research Questions and Objectives

- RQ1. *Accuracy*: Does swap-test similarity reproduce the ranking of classical cosine similarity, and what is the shot budget required to do so?
- RQ2. *Scaling*: Does our Grover implementation reproduce the  $\lfloor \pi\sqrt{N}/4 \rfloor$  oracle-call scaling in practice?
- RQ3. *Practicality*: Under what conditions, if any, would a quantum vector-search engine outperform the HNSW classical baseline on the same dataset?
- RQ4. *Fit*: Which part of cross-modal vector search, if any, is a good match for quantum computation once data loading and classical baselines are included?

To answer these questions we set four objectives, each corresponding to a concrete project output:

1. Implement a suite of search engines spanning classical, quantum, and hybrid approaches behind a single common interface, so that one query can be answered by any of them and the results compared directly.
2. Build a reproducible benchmark harness and a controlled cross-modal dataset, and

collect a complete set of results across embedding dimensions and measurement shot counts.

3. Provide a web application that displays the engines' rankings side by side for inspection.
4. Write a balanced analysis that answers the research questions from the measured data, including the cases where quantum methods do not help.



## 2. LITERATURE REVIEW

### 2.1 Classical Vector Search

The baseline for vector search is an exact scan: the query vector is compared against every stored vector, and the closest are returned. This guarantees the correct nearest neighbours but costs  $O(N)$  comparisons per query, which becomes the bottleneck as a collection grows. Facebook AI Similarity Search (FAISS) is a widely used library that performs this exact search efficiently by exploiting the vectorised instructions of modern processors, and extends it to very large collections [3]. When exact search is too slow, approximate nearest-neighbour methods accept a small loss in recall for a large gain in speed. The most widely used of these is the Hierarchical Navigable Small World (HNSW) graph, which links each vector to a set of near neighbours across several layers and answers a query by greedily descending the graph, reaching a near neighbour in roughly  $O(\log N)$  steps [4]. We use HNSW as the strongest classical baseline in this report because it is the kind of fast, approximate index a practical retrieval system would choose before considering a quantum alternative. A quantum search that hopes to be useful must therefore improve on a classical method that is already sublinear, not only on a simple linear scan.

### 2.2 Cross-Modal Embeddings

Cross-modal retrieval depends on embedding different kinds of media into a common space. CLIP (Contrastive Language-Image Pre-training) does this for text and images by training a text encoder and an image encoder together, so that an image and its caption map to nearby vectors while mismatched pairs are pushed apart [2]. The variant we use pairs a Vision Transformer image encoder (ViT-B/32) with a text encoder and produces 512-dimensional embeddings, in which the cosine similarity between a text vector and an image vector measures their relevance. This reduces text-to-image retrieval to ordinary nearest-neighbour search in a single vector space. For the quantum engines, which encode a vector into the amplitudes of a quantum state, we truncate these 512-dimensional embeddings to 64, 128, or 256 components to keep the number of simulated qubits manageable, a trade-off examined in the methodology and results chapters.

## 2.3 Quantum Similarity Primitives

Two quantum building blocks are relevant to similarity search. Grover’s algorithm searches an unstructured collection of  $N$  items for one that satisfies a condition checked by an oracle, and it does so with about  $\lfloor (\pi/4)\sqrt{N} \rfloor$  oracle queries, a quadratic improvement on the  $N$  checks a classical scan requires [5]. The swap test, introduced in work on quantum fingerprinting, compares two quantum states by placing an ancilla qubit in superposition, conditionally swapping the two states, and measuring the ancilla; the probability of reading the ancilla as zero is  $(1 + |\langle \psi | \phi \rangle|^2)/2$ , so the test recovers the squared overlap of the states and with it a similarity score [6].

Both techniques assume the data is already present as quantum states, and this assumption carries the cost that the asymptotic figures hide. Encoding  $N$  vectors into a quantum state, or building an oracle that recognises the closest of them, requires reading all  $N$  items. A quantum random-access memory has been proposed to supply data items in superposition [8], but no such device exists at usable scale, and if the data must instead be loaded one item at a time, the  $O(N)$  preparation cost cancels Grover’s quadratic advantage at the level of the whole system [7]. Hoeffler, Häner, and Troyer make the same practical point more broadly: quantum computers are most promising for small-data problems with very large computation, while big-data search is limited by input and output movement [1]. The present work therefore treats Grover’s scaling as a property of the search step in isolation and is explicit about the data-loading cost it does not include, a distinction returned to in the results chapter.

## 2.4 Hybrid Classical–Quantum Architectures

The retrieve-and-rerank cascade is a standard pattern in information retrieval: a fast, inexpensive first stage selects a small set of candidates from the whole collection, and a slower but more accurate second stage reorders only those candidates. The cost of the expensive stage is then bounded by the size of the shortlist rather than by the size of the collection. This pattern is a natural fit for near-term quantum hardware. Quantum machine learning remains constrained by considerable hardware and software challenges [9], so a practical near-term design keeps the global, data-bound work on classical hardware and reserves the quantum processor for a small, structured subproblem.

Our `hybrid_hnsw_swap_test` engine is an instance of this cascade. The classical HNSW index of Section 2.1 selects a small candidate pool in roughly  $O(\log N)$  steps, and the swap test of Section 2.3 reranks only those candidates by their estimated

similarity, executed on the Qiskit Aer simulator [10]. Because the swap test runs one circuit per candidate, confining it to the shortlist keeps the number of circuit executions independent of the collection size, at  $O(\log N + M)$  for a shortlist of  $M$  candidates. This places the quantum step where its cost is bounded and where an estimation error can only reorder an already-relevant shortlist rather than miss a relevant item entirely, an arrangement whose accuracy and cost we evaluate in the results chapter.

## 3. RESEARCH METHODOLOGY

### 3.1 System Overview

The system is a modular monolith for cross-modal retrieval. A text query and the dataset images are embedded into the same vector space, then one interchangeable engine ranks the image vectors closest to the query vector. Classical, quantum, and hybrid engines all implement the same `SearchEngineStrategy` interface, with `build_index` and `search` as the required operations. The rest of the harness, data loading, embedding, query execution, and result storage, is therefore identical for every engine.<sup>1</sup>

Figure 1 shows the five phases. A `DirectoryDataLoader` reads the images and ground-truth query pairs; CLIP embeds the images and text; the selected engine builds an index and answers the query; PostgreSQL records accuracy and resource measurements; FastAPI and React surface the rankings. The experiment is configured in `benchmarks.yaml`, where the active engines, dimensions, and shot counts are selected without editing source code.

### 3.2 Methodology Diagram

The experiment follows the script sequence in Figure 2. The import script downloads the fixed Flickr30k subset and writes the ground-truth query file. The indexing script encodes each image with CLIP and stores one vector per image in PostgreSQL. The benchmark script then iterates over every enabled engine, vector dimension, shot count, and query. Quantum and hybrid engines run on Qiskit’s noiseless simulator, so the sweep measures algorithm behaviour and finite-shot sampling rather than device noise.

Each run is stored as one row in `benchmark_results`, including MRR, oracle calls, circuit depth, and qubit count where applicable. The IBM validation script executes the hybrid swap test once on real hardware at two dimensions, two candidates, and 32 shots. This result is tagged with the IBM engine ID and is treated as a validation run, not as part of the normal sweep. Finally, the report-generation script produces `benchmark_report.md` directly from the database; Chapter 4 uses that generated file as the source of truth for all reported numbers.

---

<sup>1</sup>The full source code, benchmark harness, and dataset configuration are publicly available at <https://github.com/Mirza404/quantum-vector-search>.

### 3.3 Dataset and Ground Truth

The evaluation set is a fixed 20-image subset of Flickr30k, an image-caption corpus in which each photograph has several human-written captions. The importer requests a fixed configuration and split, then reads the first twenty rows in the dataset viewer order, with no random sampling or shuffling. A different machine therefore regenerates the same images and queries. Twenty images is small for information retrieval, but it keeps the full seven-engine simulator sweep tractable on a CPU; the separate IBM hardware validation is excluded from this count because it is a tiny real-device sanity check.

For each image we use the first caption as the query text and record the image’s Flickr identifier as the target. The result is a one-to-one evaluation: every query has exactly one correct image. This choice ignores alternate captions and cannot reward a different image that a human might also consider relevant, but it makes scoring deterministic. An engine ranks all twenty images, and MRR assigns full credit when the target is first and partial credit when it appears lower.

### 3.4 Embedding Pipeline

CLIP embeds both images and text into one vision-language space [2]. It is trained contrastively so matching images and captions are close while mismatches are pushed apart [2]. We use ViT-B/32, which produces 512-dimensional vectors, and L2-normalise every image and query vector. On unit vectors, cosine similarity, Euclidean distance, and the squared inner product estimated by the swap test induce the same ranking, which lets MRR be compared across engines that compute different similarity quantities.

The benchmark uses truncated dimensions of 64, 128, and 256 rather than the full 512. The reason is simulation cost. Amplitude encoding a  $d$ -dimensional vector requires  $\lceil \log_2 d \rceil$  qubits, and the swap test needs two such registers plus one ancilla, or  $1 + 2\lceil \log_2 d \rceil$  qubits. At 512 dimensions the swap test would need nineteen qubits and more than half a million state-vector amplitudes per comparison; at 64 dimensions it needs thirteen qubits and roughly eight thousand amplitudes. Truncation reduces simulator cost but discards information, so the Results chapter measures the accuracy loss. Engines that rely on unit length re-normalise after truncation.

## 3.5 Search Engines

### 3.5.1 Classical Baselines

#### Brute-Force Cosine

The reference engine compares the query with every image. After normalisation, cosine similarity is the dot product, so the engine computes twenty dot products, sorts the scores, and returns the top results. Its work grows as  $O(N)$ , but it cannot skip the closest image. We use its ordering as the reference ranking for the other engines.

#### FAISS Flat (Exact)

FAISS Flat delegates the same exhaustive search to FAISS, a similarity-search library covering exact, approximate, and compressed-domain search [3]. We use its exact L2 index. For unit vectors, L2 distance and cosine similarity are linked by  $\|a - b\|^2 = 2 - 2\cos\theta$ , so the ranking matches brute-force cosine after re-normalisation. FAISS Flat is therefore a production-grade exact baseline that should reproduce the reference ranking.

#### FAISS HNSW (Approximate)

FAISS HNSW uses a Hierarchical Navigable Small World graph, an approximate nearest-neighbour index that searches a layered proximity graph instead of the whole collection [4]. Its expected query cost is  $O(\log N)$ , with a recall trade-off at large scale. On our 20-image collection the graph is too small for the approximation to matter, so HNSW reproduces the exact ranking. Its role is to place the quantum engines next to the classical index a real retrieval system would use.

### 3.5.2 Quantum Engines

#### Qiskit Swap Test

The swap test estimates the squared inner product of two amplitude-encoded vectors  $|\psi\rangle$  and  $|\phi\rangle$  via a single ancilla measurement:

$$P(\text{ancilla} = 0) = \frac{1 + |\langle\psi|\phi\rangle|^2}{2}. \quad (1)$$

The procedure originates in quantum fingerprinting, where it distinguishes two unknown quantum states with high probability [6]. In our engine each vector is

amplitude-encoded into a register, and one ancilla qubit controls a swap between the registers. Figure 3 shows a four-dimensional example. Equation 1 converts the probability of measuring the ancilla as zero into an overlap score. Because that probability is estimated from a finite number of shots, the score has sampling error, so we sweep the shot count. The swap test compares one query with one candidate; a full ranking still needs  $O(N)$  circuits. We run the sweep on Qiskit’s Aer simulator [10], with a separate small IBM hardware validation.

### **Qiskit Grover (Hardcoded Oracle)**

Grover’s algorithm finds a marked item in an unstructured collection using about  $\sqrt{N}$  oracle queries, a quadratic improvement over a classical scan [5]. Our engine pads  $N$  to a power of two, prepares a uniform index superposition, then repeats an oracle and diffusion operator  $\lfloor \pi\sqrt{N}/4 \rfloor$  times before measuring. The oracle marks the target index, and the diffusion step amplifies it; Figure 4 shows one iteration for four items.

The limitation is the oracle. A real vector-search oracle would need to decide, in superposition, which stored vector is closest to the query, which requires quantum random access memory (qRAM) for loading data in superposition [8]. Practical qRAM does not exist, and loading vectors one by one costs  $O(N)$ , removing the advantage [7]. We therefore hardcode the nearest-neighbour index classically and test only the Grover search step. The claim being tested is oracle-call scaling, not end-to-end quantum speedup.

### **Qiskit Grover with Quantum State Preparation**

The quantum-preparation Grover variant chooses the target index with quantum measurements instead of a classical dot product. Each stored vector and query are loaded with Qiskit state-preparation gates; a swap test estimates each overlap; the highest measured overlap becomes the index marked by the Grover oracle. The remaining Grover circuit is unchanged. This makes target selection noisy: state-preparation circuits are deep, the swap tests have shot noise, and Grover amplifies whichever index it is given. The Results chapter shows that this variant is the least accurate engine.

### **Hybrid HNSW + Swap Test**

The hybrid engine uses HNSW to shortlist candidates, then applies the swap test only to that fixed-size pool. The classical stage performs the search; the quantum stage reranks the survivors. This mirrors common retrieval pipelines in which a fast approximate index proposes candidates and a slower scorer refines them. Because the swap test is not a search algorithm, this is the most plausible place for it in the system. On our small

dataset the shortlist usually already contains the target, so the hybrid tracks the classical baseline closely.

### IBM Hardware Validation

To check that the quantum path is not only a simulator-only result, we ran the hybrid engine once through IBM Quantum runtime. The run uses two-dimensional vectors, two candidates, and thirty-two shots, small enough for the free-tier device time and shallow enough for noisy hardware. It is a sanity check, not a scaling experiment. The Results chapter reports low accuracy, as expected from aggressive truncation, few shots, and real-device noise. We draw no scaling conclusions from it.

## 3.6 Evaluation Metrics

The primary accuracy measure is mean reciprocal rank (MRR). For each query, the engine receives the reciprocal of the rank at which the correct image appears: rank one scores one, rank two scores one half, and a target outside the top  $k$  results, with  $k = 10$ , scores zero. Averaging over all queries gives Equation 2. Because every engine is scored against the same target image, MRR is comparable across engines.

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q}, \quad (2)$$

where  $\text{rank}_q$  is the position of the first relevant result for query  $q \in Q$  and  $1/\text{rank}_q := 0$  if no relevant result is in the top- $k$ .

We also store operation count, the number of core comparisons, graph steps, or oracle calls per query. Brute force, FAISS flat, and the standalone swap test scale as  $O(N)$ ; HNSW as  $O(\log N)$ ; the hybrid as  $O(\log N + M)$ ; and Grover as  $O(\sqrt{N})$  with  $\lfloor \pi\sqrt{N}/4 \rfloor$  oracle cycles. This count is not tied to any particular hardware, so it is the only speed-like quantity compared across engines.

For circuit engines we record depth and qubit count. Depth is a proxy for noise exposure on real hardware, while qubit count is the circuit width. We do not compare wall-clock time across engines because the quantum engines run on a classical simulator whose runtime reflects state-vector simulation overhead. Wall-clock appears only within an engine, where the hardware and simulator are held fixed.



### 3.7 Experimental Configuration

Table 2 lists the sweep. Each engine is evaluated at every vector dimension; quantum and hybrid engines also run at every shot count. With twenty queries, this gives 900 simulator runs: three classical configurations and twelve quantum configurations per dimension, across three dimensions. The IBM hardware validation adds 20 rows, for a stored total of 920 results. The layers parameter is fixed at one because the harness requires it, although no enabled engine uses it. Results are stored in PostgreSQL and dumped to the committed seed file.

**Table 2:** Benchmark configuration grid.

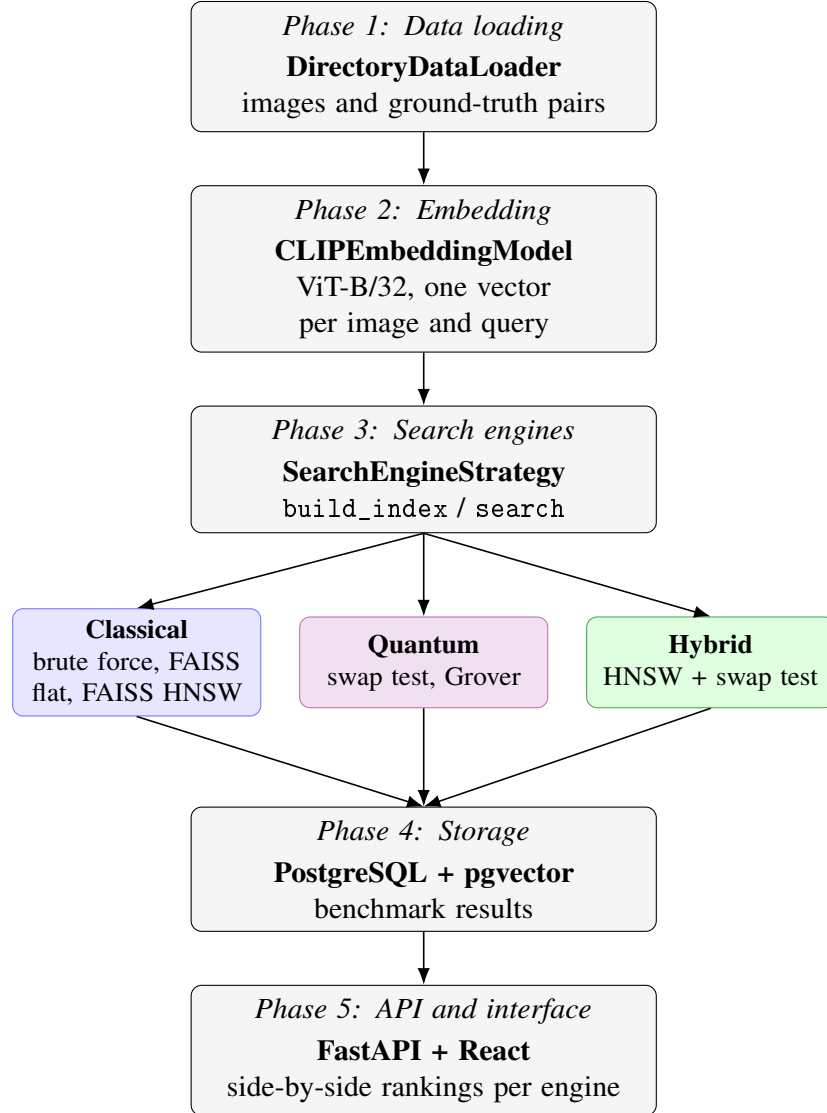
| Parameter         | Values                                                      |
|-------------------|-------------------------------------------------------------|
| Classical engines | Brute-force cosine; FAISS flat; FAISS HNSW                  |
| Quantum engines   | Swap test; Grover (hardcoded oracle); Grover (quantum prep) |
| Hybrid engine     | HNSW + swap test                                            |
| Dimensions        | 64, 128, 256                                                |
| Shots             | 512, 1024, 2048 (quantum engines only)                      |
| Layers            | 1                                                           |
| Top- $k$          | 10                                                          |

### Use of AI Tools

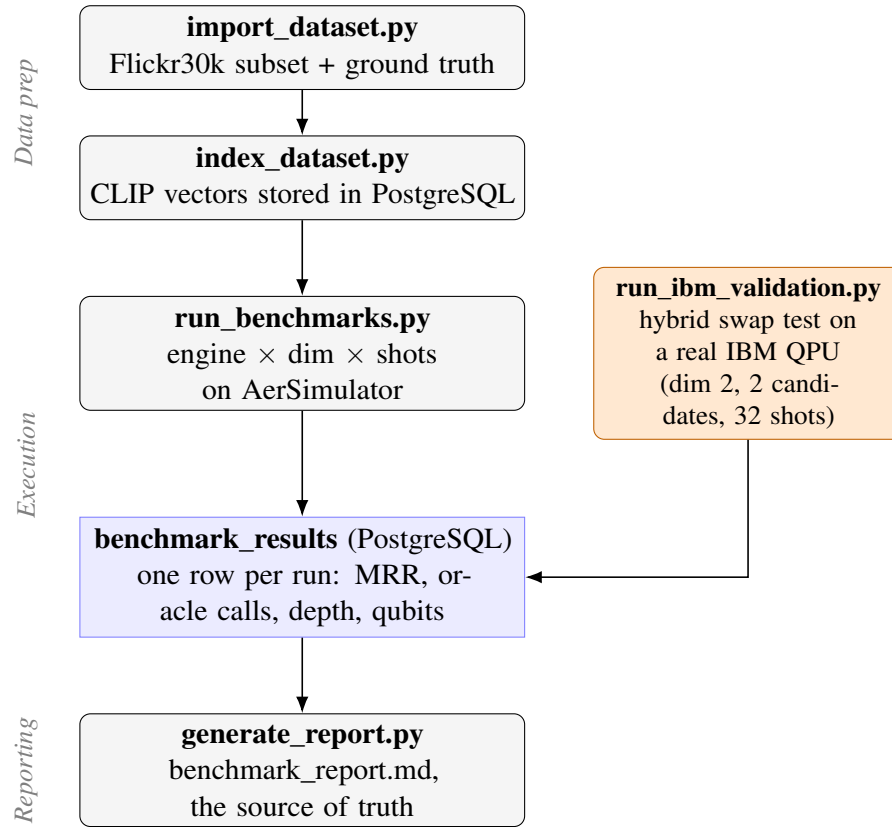
This project used AI assistants, specifically Claude Code and ChatGPT, as productivity tools during development and writing. They assisted with routine code generation and refactoring, code review, bug-location during the audit, prose drafts revised by the authors, and diagram rendering from author-specified layouts.

The intellectual core is the authors’ own: system architecture, engine selection, benchmark design, research questions, and interpretation. Every benchmark figure reported here was produced by our own runs, and every external reference was verified by the authors before citation.

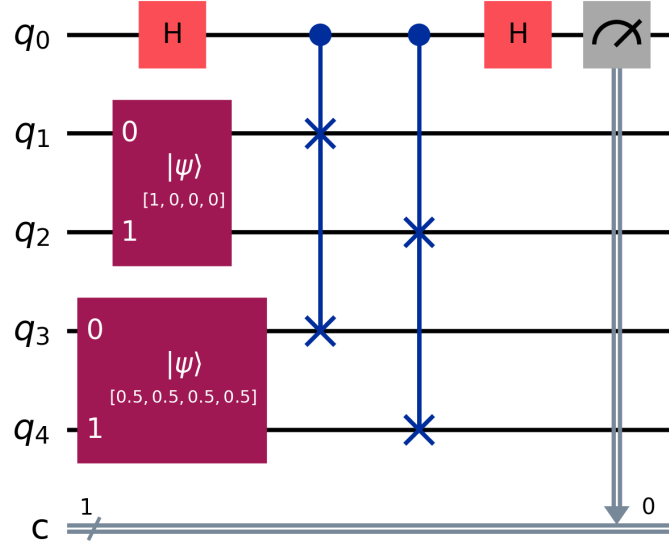
The authors take full responsibility for the correctness, originality, and verification of all code, data, and conclusions in this report. Where AI tools contributed text or code, that output was read, checked, and where necessary corrected and rewritten by the authors, who treat it as a draft to be scrutinised rather than as an authority to be trusted.



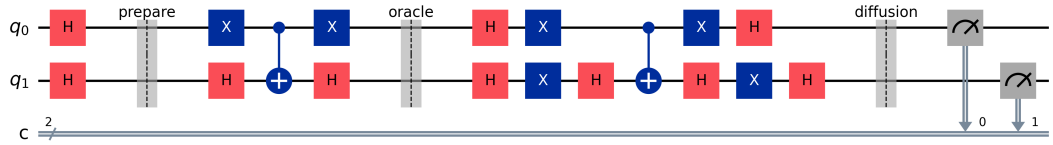
**Figure 1:** End-to-end pipeline. Data loading and CLIP embedding feed a common engine-strategy interface, behind which the classical, quantum, and hybrid engines are interchangeable. Every run is recorded in PostgreSQL and surfaced through the FastAPI and React interface.



**Figure 2:** Methodology flow. Four scripts run in order, from dataset import through CLIP indexing, the simulator benchmark sweep, and report generation. The IBM hardware validation is a separate run that writes its single result into the same database table.



**Figure 3:** Swap-test circuit for a four-dimensional example, in which each vector is amplitude-encoded into two qubits. The ancilla ( $q_0$ ) is placed in superposition, controls a swap of the two encoded registers ( $q_1q_2$  and  $q_3q_4$ ), and is measured after a second Hadamard. The probability of reading the ancilla as zero gives the squared overlap of the two states, as in Equation 1.



**Figure 4:** One Grover iteration for a four-element index ( $N = 4$ , two index qubits). After preparing a uniform superposition, the oracle flips the phase of the marked index and the diffusion operator amplifies its amplitude; the two qubits are then measured. The barriers separate the preparation, oracle, and diffusion phases.

## 4. RESULTS AND ANALYSIS

In the noiseless simulator benchmark, retrieval accuracy mainly depended on the embedding dimension, not on whether the engine was classical or quantum. The three classical baselines returned identical rankings and tied at MRR 0.829, and every engine improved as the dimension rose from 64 to 256. The swap test reproduced the classical ranking at dimension 256 (MRR 0.975) and recovered at lower dimensions as shots increased. Grover with a hardcoded oracle matched the theoretical  $\lfloor (\pi/4)\sqrt{N} \rfloor$  oracle count, but its probabilistic measurement stopped at MRR 0.955 at dimension 256. Grover with quantum state preparation was weakest (MRR 0.559). The hybrid engine effectively tied the classical baseline (MRR 0.823) by using HNSW for search and the swap test only for reranking.

These accuracy results should not be read as predictions for a large, noisy quantum computer. The simulator removes gate noise, measurement error beyond finite-shot sampling, queueing, and hardware latency. The measured pattern still answers the algorithmic question under controlled conditions, but real hardware would be expected to degrade the quantum-engine accuracy unless error rates and circuit depth are low enough for the chosen workload.

### 4.1 Accuracy Across Engines

**Table 3:** Average Mean Reciprocal Rank (MRR) by engine and embedding dimension, over the twenty benchmark queries. Higher is better: an MRR of 1.000 means the correct image was ranked first for every query. Quantum cells average over the three shot counts (512, 1024, 2048); the shot dependence is examined separately below. The IBM hardware run is reported in Section 4.4.

| Engine                      | MRR by dimension |       |       |       |
|-----------------------------|------------------|-------|-------|-------|
|                             | 64               | 128   | 256   | Avg   |
| Brute-force cosine          | 0.655            | 0.858 | 0.975 | 0.829 |
| FAISS flat                  | 0.655            | 0.858 | 0.975 | 0.829 |
| FAISS HNSW                  | 0.655            | 0.858 | 0.975 | 0.829 |
| Swap test                   | 0.470            | 0.738 | 0.975 | 0.728 |
| Grover (hardcoded oracle)   | 0.546            | 0.776 | 0.955 | 0.759 |
| Grover (quantum state prep) | 0.404            | 0.551 | 0.722 | 0.559 |
| Hybrid (HNSW + swap test)   | 0.635            | 0.858 | 0.975 | 0.823 |

Table 3 shows that brute-force cosine, FAISS flat, and FAISS HNSW all scored MRR 0.829 and produced the same query-by-query ordering. This is expected: brute force and FAISS flat are exact searches over the same twenty unit vectors, and HNSW’s approximation does not matter on a graph this small. We treat the brute-force result as the reference ordering.

Embedding dimension had the strongest effect on accuracy. Every engine improved from dimension 64 to 256, and the classical baseline moved through 0.655, 0.858, and 0.975. Truncating CLIP vectors discards information that separates near-neighbours; restoring dimensions recovers it. The lower dimensions are therefore a simulation trade-off, not the configuration a retrieval system would prefer.

The swap test illustrates how a quantum estimator approaches the exact answer as its measurement budget grows. At dimension 256 it matched the best classical score exactly (MRR 0.975 at every shot count), because the overlaps it estimates are well separated and even a coarse estimate ranks them correctly. At dimension 128, where neighbours are closer together, accuracy depended on how finely we measured: MRR rose from 0.642 at 512 shots to 0.688 at 1024 and 0.883 at 2048. The swap test estimates the squared overlap  $|\langle \psi | \phi \rangle|^2$  from a finite number of measurements, so additional shots reduce the sampling noise and sharpen the ranking. This shot-versus-accuracy trade-off, rather than a fixed limit of the method itself, explains the swap test’s lower average of 0.728: the average is pulled down by the low-dimension, low-shot configurations that a practical deployment would not choose.

The two Grover engines separate sharply. The hardcoded-oracle version reached MRR 0.955 at dimension 256, just short of the best classical score of 0.975, because the marked state is still measured probabilistically. The quantum state-preparation variant was weakest at every dimension and averaged 0.559: state preparation and per-candidate swap tests add noise before Grover amplifies the chosen index. The hybrid takes the opposite approach. HNSW chooses the candidates, and the swap test reranks only that shortlist, so it stayed close to the baseline (MRR 0.823, and 0.975 at dimension 256).

## 4.2 Quantum Resource Cost

Because the quantum engines run on a classical simulator, their wall-clock time measures simulation overhead rather than the cost of a real quantum device (Section 4.3 returns to this). The hardware-neutral way to compare the engines is therefore to count the operations each one performs per query and the resources its circuit would demand on a real device. Table 4 reports the first of these: the number of similarity comparisons

or oracle calls each engine makes for a single query over the twenty-image dataset.

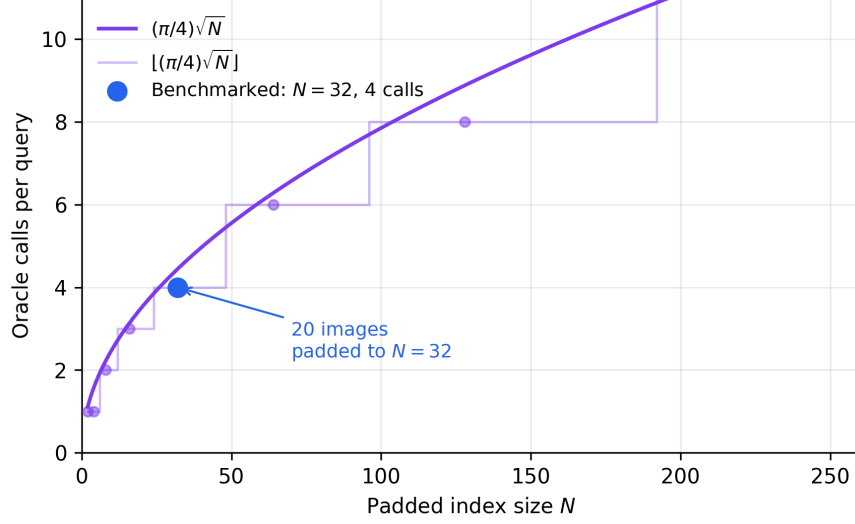
**Table 4:** Operations per query and algorithmic complexity for each engine over the twenty-image dataset ( $N = 20$ ). The operation count is the cross-engine scaling metric; it is independent of hardware. Here  $M$  denotes the number of candidates the hybrid engine reranks.

| Engine                      | Operations / query | Complexity          |
|-----------------------------|--------------------|---------------------|
| Brute-force cosine          | 20                 | $O(N)$              |
| FAISS flat                  | 20                 | $O(N)$              |
| FAISS HNSW                  | 5                  | $O(\log N)$ approx. |
| Swap test                   | 20                 | $O(N)$              |
| Grover (hardcoded oracle)   | 4                  | $O(\sqrt{N})$       |
| Grover (quantum state prep) | 4                  | $O(\sqrt{N})$       |
| Hybrid (HNSW + swap test)   | 10                 | $O(\log N + M)$     |

Grover was the only engine whose per-query cost was sublinear in the dataset size. The exact classical engines and the standalone swap test each performed one comparison per stored vector, twenty in total, while FAISS HNSW reached its answer in about five graph hops and the hybrid engine made ten comparisons (one HNSW shortlist followed by a swap-test rerank of the survivors). Both Grover engines used four oracle calls, and this count is exactly what the theory predicts once the index padding is accounted for. Grover addresses an index register whose length is a power of two, so the twenty images are padded to the next power of two,  $N' = 32$ , which is why the Grover circuits used five qubits ( $\lceil \log_2 32 \rceil = 5$ ). The optimal number of Grover iterations for an unstructured search of  $N'$  items is  $\lfloor (\pi/4)\sqrt{N'} \rfloor$ , and for  $N' = 32$  this evaluates to  $\lfloor 4.44 \rfloor = 4$ , matching the measured count. Figure 5 places this verified point on the theoretical curve.

The two quantum families differ sharply in what they encode, and Table 5 shows the consequence for circuit width and depth. Grover encodes the candidate *index*, so its width stayed fixed at five qubits regardless of the embedding dimension; its cost instead appears as depth, around fifty gate layers, because each of the four iterations applies an oracle and a diffusion operator in sequence. The swap test and the hybrid engine encode the candidate *vectors*, so their width grew with the embedding dimension as  $1 + 2\lceil \log_2 d \rceil$  qubits, from thirteen at dimension 64 to seventeen at dimension 256, while their circuits stayed shallow (depth nine to eleven). On near-term hardware both costs matter: more qubits are harder to allocate, and greater depth gives decoherence more time to corrupt the state, which is the practical reason the deep Grover circuits would be the harder of the two to run faithfully on a real device.

These counts also expose the assumption hidden inside Grover’s sublinear



**Figure 5:** Theoretical Grover oracle-call count  $\lfloor (\pi/4)\sqrt{N} \rfloor$  as a function of the padded index size  $N$ , with a marker at the single size we benchmarked ( $N = 32$ , four oracle calls). We ran one dataset size, so this confirms the count at a single point on the curve rather than tracing the curve empirically.

**Table 5:** Circuit width and depth for the quantum engines, from the compiled circuits. The swap test and the hybrid engine share the same circuit metrics because the hybrid reuses the swap-test circuit. Grover’s width is set by the padded index register, not the embedding dimension, so it is constant.

| Engine (dimension)              | Qubits | Circuit depth |
|---------------------------------|--------|---------------|
| Swap test / Hybrid, dim 64      | 13     | 9             |
| Swap test / Hybrid, dim 128     | 15     | 10            |
| Swap test / Hybrid, dim 256     | 17     | 11            |
| Grover (both variants), any dim | 5      | $\approx 50$  |

figure. The four oracle calls measure only the search once the data is already encoded in the oracle; they do not include the cost of building that oracle. Our hardcoded oracle computes the nearest neighbour classically and then marks it, so the twenty comparisons Grover appears to save have in effect been paid before the circuit runs. This is the qRAM caveat set out in Section 3.5: a genuine quantum speed-up for search would require loading the dataset into a quantum-accessible memory, which is itself an unsolved cost. The swap test and hybrid engines make the opposite trade, paying their cost openly in qubits and in one circuit execution per candidate. Section 4.5 returns to what this means for the practicality question.



### 4.3 Wall-Clock Behaviour (Within-Engine Only)

Wall-clock time cannot be compared across engines in this study, and Table 6 should be read only one row at a time. The reason is that the quantum engines did not run on a quantum computer: they ran on a classical statevector simulator, which represents the full quantum state in ordinary memory and so does work that grows exponentially with the number of qubits. The times for the quantum engines therefore measure the cost of *simulating* a circuit on a CPU, not the latency a real quantum processor would incur, and putting them next to the classical millisecond figures would compare two unrelated quantities. The within-engine comparison across dimension is meaningful, because there the hardware is held fixed and only the problem size changes.

**Table 6:** Mean total time per query, in milliseconds, by engine and embedding dimension. The three classical engines completed in well under a millisecond (0.2 to 0.6 ms) and are shown as “< 1”. The quantum and hybrid times are simulator times on a CPU and are *not* comparable to the classical figures or to real quantum hardware; they are reported only to show how each engine’s simulation cost scales with dimension.

| Engine                      | Mean time per query (ms) |         |         |
|-----------------------------|--------------------------|---------|---------|
|                             | dim 64                   | dim 128 | dim 256 |
| Brute-force cosine          | < 1                      | < 1     | < 1     |
| FAISS flat                  | < 1                      | < 1     | < 1     |
| FAISS HNSW                  | < 1                      | < 1     | < 1     |
| Swap test                   | 993                      | 4764    | 21555   |
| Grover (hardcoded oracle)   | 19                       | 20      | 19      |
| Grover (quantum state prep) | 3399                     | 6916    | 7453    |
| Hybrid (HNSW + swap test)   | 254                      | 1134    | 5255    |

Read row by row, the table reflects circuit size. The swap test and hybrid slowed sharply as dimension rose, because each doubling adds two qubits to the amplitude encoding and quadruples the simulator state. The swap test grew from about one second per query at dimension 64 to roughly twenty-two seconds at dimension 256. Hardcoded Grover stayed near twenty milliseconds because it uses the same five-qubit index register at every dimension. The simulator, not the algorithm, set the cap of 256 dimensions.

### 4.4 IBM Hardware Sanity Check

All of the quantum results above were produced on a simulator. To confirm that the swap-test pipeline also runs on a physical quantum processor, we executed the hybrid engine once on real IBM Quantum hardware through the IBM Quantum cloud

service. This run was deliberately tiny: it used two-dimensional vectors, two candidates per query, and thirty-two measurement shots, across the same twenty queries, and it consumed eighty seconds of the limited free-tier quota. We treat it as a sanity check on the execution path, not as a benchmark, and it is kept separate from the simulator results for that reason.

The run achieved MRR 0.125, far below the simulator figures. This reflects the configuration rather than the method: two-dimensional vectors remove most semantic information, two candidates make the task close to a coin toss, and thirty-two shots give a noisy overlap estimate before device noise is added. The run establishes only that the hardware path works end to end. It does not support accuracy or scaling claims.

## 4.5 Discussion

**RQ1 (Accuracy).** The swap test reproduced the retrieval accuracy of classical cosine similarity once it was given enough measurement shots, and the shot budget it needed depended on how close the candidate vectors were. At dimension 256, where neighbours are well separated, it matched the classical MRR of 0.975 at every shot count we tried, including the smallest. At dimension 128 it reached classical accuracy only at the highest setting, rising to 0.883 at 2048 shots from 0.642 at 512, and at dimension 64 it remained below the classical baseline even at 2048 shots. The pattern is consistent with the swap test being a sampled estimate of vector overlap: the harder the discrimination, the more shots are required to resolve it. *Yes: the swap test reproduces classical retrieval accuracy given a sufficient shot budget, and the budget required grows as the embedding dimension, and hence the separation between neighbours, falls.*

**RQ2 (Scaling).** Our Grover implementation used four oracle calls per query, which is exactly the theoretical optimum  $\lfloor (\pi/4)\sqrt{N'} \rfloor$  for the padded index size  $N' = 32$ , as set out in Section 4.2. The match was exact rather than approximate. The qualification is that we benchmarked only one dataset size, so we confirmed the formula at a single point rather than tracing the scaling curve across a range of  $N$ . *Yes: at the size we tested, the oracle-call count matched the theoretical  $\lfloor (\pi/4)\sqrt{N} \rfloor$  optimum exactly, though a single dataset size cannot demonstrate the full curve.*

**RQ3 (Practicality).** No engine that performed the search on a quantum circuit beat the classical HNSW baseline on this dataset. HNSW reached the full brute-force accuracy of 0.829 in about five graph hops, while the standalone quantum engines were either less accurate or, in Grover’s case, dependent on a classically precomputed oracle. The one quantum-containing engine that matched the baseline, the hybrid, did

so precisely because the classical index performed the search and the swap test only reranked a shortlist; its accuracy is inherited from HNSW, not produced by the quantum step. *We found no condition under which a quantum search engine outperformed the HNSW baseline on this dataset; the strongest quantum-containing engine matched it only by delegating the search itself to the classical index.*

**RQ4 (Fit).** The resource accounting in Section 4.2 points to a reason for the RQ3 result. Grover’s sublinear oracle count is genuine, but it measures the search after the data has been loaded into the oracle; constructing that oracle, or an equivalent quantum-accessible memory, reintroduces a cost linear in the dataset size. Vector retrieval is dominated by exactly this data-loading step, because answering a query means comparing against many stored items. Quantum advantage, by contrast, is generally argued to concentrate on compute-bound problems that do heavy processing over small inputs rather than on data-bound problems that must read a large dataset [1, 7]. *On both our evidence and the structure of the problem, the search part of cross-modal vector retrieval is not a good match for quantum advantage under the methods we tested. A quantum step is more plausible as a reranker for a small shortlist than as the engine that scans or indexes the whole dataset.*

## 4.6 Limitations

Several constraints bound the results. The dataset has twenty images and one query per image, which kept the sweep simulable and the target unambiguous but limited statistical power. HNSW did not have enough vectors to exercise its approximation, so its match with exact search holds only here. Except for the IBM sanity check, the quantum engines ran on a classical simulator, so the results do not measure present-day hardware noise or latency. Embeddings were truncated from 512 to at most 256 dimensions to fit the qubit budget, so absolute MRR values are comparisons under a shared handicap. Grover used a classically precomputed oracle, and its oracle-call scaling was checked at one dataset size, not across a range.

## 5. SUSTAINABILITY, INCLUSIVITY, AND SOCIETAL IMPACT

This project was designed to be modest in its use of resources. The quantum engines ran on a CPU-based circuit simulator over a deliberately small dataset of twenty images, rather than on large-scale GPU training or extended quantum-hardware time, so the entire benchmark sweep completes on a single workstation; the one run on real quantum hardware consumed eighty seconds of a free-tier quota. The data raises no privacy concerns, as Flickr30k is a public research dataset of captioned images and contains no personal identifying information that we collect or process. The work is openly reproducible: the source code, the benchmark harness, and the dataset configuration are publicly available, and the harness is deterministic, so the reported results can be regenerated from scratch.

Beyond these practical points, the study makes a small contribution to how quantum computing is discussed. Practical-advantage analyses warn that quantum speed-ups are limited by data movement and by the size of the speed-up itself [1]; this project shows that issue in a concrete retrieval system. By measuring where quantum search helps and where it does not, the work offers a careful explanation rather than a general claim that quantum methods are always useful. In the language of the United Nations Sustainable Development Goals, the project speaks most directly to SDG 4 (quality education), as an openly available teaching resource for quantum and classical retrieval, and to SDG 9 (industry, innovation, and infrastructure), as an evidence-based assessment of a candidate technology’s readiness.

## 6. CONCLUSION

This study built a single framework in which classical, quantum, and hybrid engines answer the same cross-modal queries over the same embeddings, and used it to measure where quantum similarity search helps. In the simulator benchmark, retrieval accuracy mainly depended on the embedding dimension rather than on whether an engine was quantum: the swap test reproduced classical accuracy once given a sufficient measurement budget (RQ1), and our Grover implementation reproduced its theoretical  $\lceil (\pi/4)\sqrt{N} \rceil$  oracle-call count exactly at the size we tested (RQ2). No engine that performed the search on a quantum circuit outperformed the classical HNSW baseline on this dataset; the only quantum-containing engine to match it, the hybrid, did so by delegating the search to the classical index (RQ3). The reason is that vector retrieval is dominated by reading the dataset, while quantum advantage is most plausible when a small amount of data requires very large computation. Under the methods we tested, cross-modal search is therefore not a strong target for a quantum search engine, although a quantum reranker can still be studied as a bounded second stage (RQ4).

This work delivers four things. The first is a reproducible benchmarking framework that runs seven engines, classical, quantum, and hybrid, behind one interface and records every result in a database. The second is an empirical verification of textbook quantum behaviour on a real retrieval task: the swap test’s convergence to the classical ranking, and Grover’s oracle-call scaling. The third is a web application that displays the engines’ rankings side by side, making the comparison inspectable rather than abstract. The fourth, and the one we consider most important, is the honest framing itself: stating plainly where quantum search does not help, and why, is a contribution in a setting where that question is often left unasked.

Several directions would extend the study. A larger dataset, drawn from the full Flickr30k set or an ImageNet subset, would exercise the HNSW approximation that twenty vectors leave untested and give the accuracy figures more statistical weight. A wider dimension sweep, and a study of variational state-preparation circuits in place of the hardcoded oracle, would probe the quantum engines closer to their intended use. Comparing against a production vector store such as pgvector’s HNSW would add a third classical baseline, and a characterisation of noise on real quantum hardware, beyond the single validation run reported here, would show how the simulated behaviour degrades on present-day devices.

## REFERENCES

- [1] T. Hoefler, T. Häner, and M. Troyer, “Disentangling hype from practicality: On realistically achieving quantum advantage,” *Communications of the ACM*, vol. 66, no. 5, pp. 82–87, 2023, also available as arXiv:2307.00523.
- [2] A. Radford, J. W. Kim, C. Hallacy *et al.*, “Learning transferable visual models from natural language supervision,” in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021, pp. 8748–8763.
- [3] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [4] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [5] L. K. Grover, “A fast quantum mechanical algorithm for database search,” in *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*, 1996, pp. 212–219.
- [6] H. Buhrman, R. Cleve, J. Watrous, and R. de Wolf, “Quantum fingerprinting,” *Physical Review Letters*, vol. 87, no. 16, p. 167902, 2001.
- [7] S. Aaronson, “Read the fine print,” *Nature Physics*, vol. 11, no. 4, pp. 291–293, 2015.
- [8] V. Giovannetti, S. Lloyd, and L. Maccone, “Quantum random access memory,” *Physical Review Letters*, vol. 100, no. 16, p. 160501, 2008.
- [9] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, “Quantum machine learning,” *Nature*, vol. 549, no. 7671, pp. 195–202, 2017.
- [10] Qiskit Contributors, “Qiskit: An open-source framework for quantum computing,” 2024, <https://www.ibm.com/quantum/qiskit>.

## A. APPENDICES

### Appendix A: Engine Configuration

The benchmark sweep is driven by a single configuration file, `backend/config/benchmarks.yaml`. The active configuration used for the results in Chapter 4 is reproduced below. Commenting out an entry removes it from the sweep without changing any code; the IBM hardware engine is kept commented out so that it runs only when quantum-processor time is deliberately spent.

```
classical_engines:
  - brute_force_cosine
  - faiss_flat_l2
  - faiss_hnsw_l2

quantum_engines:
  - hybrid_hnsw_swap_test
  # - hybrid_hnsw_swap_test_ibm # IBM hardware; run separately
  - qiskit_swap_test
  - qiskit_grover
  - qiskit_grover_quantum_prep

dimensions: [64, 128, 256] # CLIP output truncated to the first d
  components
shots_values: [512, 1024, 2048]
layers_values: [1]
top_k: 10

# Live /api/search runs a single configuration, chosen for
  responsiveness.
live_search:
  dimension: 128
  shots: 512
  layers: 1
  top_k: 10
```

## Appendix B: Database Schema

Every benchmark run is stored as one row in the `benchmark_results` table. The schema below records the query and engine, the configuration (dimension, shots, layers), the returned and target identifiers used to compute the Mean Reciprocal Rank, and the cost measures discussed in Section 4.2. The unique constraint makes each (query, engine, dimension, shots, layers) combination idempotent, so re-running the sweep overwrites rather than duplicates.

```
CREATE TABLE benchmark_results (  
  id          BIGSERIAL PRIMARY KEY,  
  recorded_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  query_id    TEXT        NOT NULL,  
  engine_name TEXT        NOT NULL,  
  dimension   INTEGER     NOT NULL,  
  target_ids  JSONB       NOT NULL,  
  top_ids     JSONB       NOT NULL,  
  state_prep_ms DOUBLE PRECISION,  
  search_ms   DOUBLE PRECISION NOT NULL,  
  total_ms    DOUBLE PRECISION NOT NULL,  
  parameters  JSONB       NOT NULL DEFAULT '{}',  
  dataset_size INTEGER     NOT NULL DEFAULT 0,  
  circuit_depth INTEGER,  
  num_qubits  INTEGER,  
  shots       INTEGER     NOT NULL DEFAULT -1,  
  layers      INTEGER     NOT NULL DEFAULT -1,  
  oracle_calls INTEGER,  
  CONSTRAINT uq_run_key UNIQUE (query_id, engine_name, dimension, shots  
    , layers)  
);
```

## Appendix C: Source Code Repository

The complete source code, the benchmark harness, the dataset configuration, and the scripts that generate the figures in this report are publicly available at <https://github.com/Mirza404/quantum-vector-search>. The repository includes the instructions needed to reproduce the dataset, run the benchmark sweep, and launch the web application described in Chapter 3.